

# PRODUCT ADMIN DASHBOARD

Product Requirements Document (PRD)

Frontend Technical Assessment  
Target: Fresher / Junior React Developer  
Recommended Duration: 5 Working Days / 30-35 Hours

|                  |                                                                          |
|------------------|--------------------------------------------------------------------------|
| Document Version | 1.0                                                                      |
| Project Type     | Frontend Coding Assessment                                               |
| Primary Stack    | Next.js, React, Tailwind CSS, Axios                                      |
| API              | DummyJSON                                                                |
| Deployment       | Vercel or equivalent                                                     |
| Assessment Focus | Frontend fundamentals, API integration, async behavior, UX, code quality |

This PRD converts the original assignment brief into a structured, candidate-ready specification. AI tools are permitted, but candidates must understand and explain their submitted code.

# 1. Executive Summary

Build a responsive Product Admin Dashboard for an authenticated administrator. The application consumes the DummyJSON REST API and supports product discovery and management through login, listing, search, filtering, sorting, pagination, details, add/edit/delete, and robust loading/error/empty states.

# 2. Objectives

- Demonstrate practical React/Next.js application development.
- Demonstrate REST API integration and clean API-layer separation.
- Demonstrate authentication and protected routes.
- Demonstrate asynchronous UI behavior, including debounced search and stale-response protection.
- Demonstrate responsive UI implementation and basic accessibility.
- Demonstrate Git workflow, deployment, documentation, and ability to explain AI-assisted code.

# 3. Scope

| In Scope                                                                                                                                                                                                        | Out of Scope                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Login/logout, protected routes, product list, pagination, search, debounce, category filter, sorting, product details, add/edit/delete, validation, loading/error/empty states, responsive UI, URL query state. | Multi-role RBAC, real payment flows, persistent production database, advanced analytics, real-time collaboration, enterprise deployment infrastructure. |

# 4. Technology Requirements

| Technology              | Requirement                                    |
|-------------------------|------------------------------------------------|
| Next.js                 | Mandatory                                      |
| React                   | Mandatory                                      |
| Tailwind CSS            | Mandatory                                      |
| Axios                   | Mandatory; use a shared Axios instance         |
| JavaScript / TypeScript | Either is acceptable; TypeScript is encouraged |
| Git / GitHub            | Mandatory                                      |
| DummyJSON               | Primary API source                             |

# 5. User Role

Administrator: can authenticate, view products, search, filter, sort, paginate, view details, add, edit, delete, and logout. No multi-role permission model is required.

# 6. High-Level User Flow

Login → Authenticate → Product Dashboard → Search / Filter / Sort / Pagination → View / Add / Edit / Delete → Logout

## 7. Functional Requirements

### 7.1 Authentication

- Provide username and password fields and a Login button.
- Use DummyJSON authentication rather than hardcoding success.
- Test credentials: username emilys, password emilyspass.
- Display a clear error when authentication fails.
- Show a loading state during login and prevent duplicate submissions.
- Store authentication information according to the application's architecture and use it for protected API requests as appropriate.

### 7.2 Protected Routes and Logout

- Unauthenticated users attempting to access product pages must be redirected to Login.
- Logout must clear authentication state and return the user to Login.
- Protected pages must not remain usable after logout.

### 7.3 Product Listing

- Display product image, title, category, price, rating, stock, and actions.
- Desktop may use a table or structured grid.
- Mobile should use a responsive card or equivalent layout rather than an unusable wide table.
- Provide View, Edit, and Delete actions.

### 7.4 Pagination

- Use DummyJSON limit/skip pagination.
- Provide page-size options: 10, 20, and 50.
- Provide previous/next and page navigation.
- Current page and page size should be represented in the URL.

### 7.5 Search

- Use the DummyJSON product search endpoint.
- Debounce search input; recommended delay is 300–500 ms.
- Do not send one API request per keystroke.
- Prevent stale API responses from overwriting newer results.
- Acceptable approaches include AbortController, request sequence IDs, or an equivalent safe strategy.

### 7.6 Category Filter and Sorting

- Provide category filtering using API-supported categories.
- Provide sorting for at least title, price, and rating.
- Document any DummyJSON limitations when combining search, filtering, sorting, or pagination.
- The selected search/filter/sort state should be reflected in the URL where practical.

### 7.7 Product Details

- Provide a product detail route such as /products/[id].

- Display product images, title, description, category, price, rating, stock, and other useful API fields.
- Handle invalid/non-existent IDs with a friendly Product Not Found state.

### **7.8 Add Product**

- Provide a form for creating a product.
- Validate required fields and numeric values.
- Show saving state and prevent repeated submissions.
- Update the visible application state after a successful mock API operation.

### **7.9 Edit Product**

- Load the selected product into an editable form.
- Validate changes before submission.
- Show saving state and update visible state after a successful operation.

### **7.10 Delete Product**

- Require explicit confirmation before deletion.
- Show deleting state and prevent duplicate requests.
- Update the visible product list after a successful mock delete.

## 8. UX and State Requirements

| State            | Expected Behavior                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------|
| Loading          | Show a spinner, skeleton, or clear loading message while an async operation is running.            |
| Empty            | Clearly indicate when there are no matching products.                                              |
| Error            | Show a user-friendly error and provide Retry where appropriate.                                    |
| Success          | Reflect successful add/edit/delete actions in the current UI.                                      |
| Duplicate action | Disable or otherwise guard buttons while the request is in progress.                               |
| Invalid URL      | Normalize or safely handle invalid page, page size, search, or product ID values without crashing. |

## 9. URL State

The product-listing URL should preserve relevant state such as page, page size, search, category, and sort. Example:

```
/products?page=2&pageSize=20&search=phone&category=smartphones&sort=price
```

Refreshing or sharing the URL should reproduce the corresponding view where the API supports it.

## 10. Axios / API Architecture

Use one shared Axios setup rather than repeating base configuration in components. A suggested separation is:

```
UI Component → Product/Auth Service → Shared Axios Instance → DummyJSON API
```

Suggested project organization:

```
src/app/ • src/components/ • src/api/ • src/hooks/ • src/types/ • src/utils/
```

Components should not contain repeated Axios setup or unrelated API plumbing.

## 11. API Limitations

DummyJSON is a mock API. Mutation endpoints such as POST/PUT/DELETE should not be treated as a permanent production database. The candidate should update local application state after successful mock operations and document this limitation in the README.

If the API cannot directly support a requested combination such as search + category filtering, the candidate must choose and document a sensible UI/application behavior rather than silently implementing misleading controls.

## 12. Responsive and Accessibility Requirements

- Support mobile, tablet, and desktop layouts.
- Use semantic HTML where practical.
- Provide visible labels for form controls.
- Ensure buttons are keyboard accessible.
- Provide visible focus states.

- Use meaningful action text and accessible error messages.

## 13. AI Usage Policy

AI development tools are allowed, including ChatGPT, Claude, Gemini, GitHub Copilot, Cursor, and similar tools. The candidate remains responsible for the submitted code and must be able to explain it.

The README should briefly state where AI was used and what was verified or implemented manually. The evaluator may ask the candidate to explain, debug, or modify the submitted code.

## 14. Git Requirements

Use meaningful commits that show normal development progression. A single commit containing the entire project is discouraged.

Example progression:

- Initial Next.js setup
- Implement authentication
- Create Axios API layer
- Implement product listing
- Implement pagination
- Implement search and debounce
- Implement filters and sorting
- Implement product details
- Implement CRUD
- Add error/loading/empty states
- Fix race conditions and edge cases
- Responsive improvements
- Final testing and documentation

## 15. Deliverables

- Accessible GitHub repository with meaningful commit history.
- Working deployed application (Vercel, Netlify, or equivalent).
- README with setup, API, architecture, features, limitations, deployment, and AI usage.
- Short technical note describing key decisions, problems encountered, solutions, and AI assistance.

## 16. Acceptance Criteria

| Area           | Acceptance Criteria                                                                               |
|----------------|---------------------------------------------------------------------------------------------------|
| Authentication | Login works with provided credentials; errors display; protected pages are guarded; logout works. |
| Products       | List, details, add, edit, and delete flows work and do not crash on invalid data.                 |
| Search         | Search works, is debounced, and stale responses cannot replace newer results.                     |

| Area                | Acceptance Criteria                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------------------|
| Filtering / Sorting | Category and sorting controls work within API capabilities; limitations are documented.               |
| Pagination          | Pagination and 10/20/50 page sizes work; relevant state is reflected in URL.                          |
| UX                  | Loading, empty, error, success, confirmation, and duplicate-request states are handled.               |
| Responsive          | Application is usable on mobile, tablet, and desktop.                                                 |
| Engineering         | Shared Axios setup, reasonable component separation, Git history, deployment, and README are present. |

## 17. Recommended 5-Day Timeline

| Day   | Focus              | Expected Output                                                                                 |
|-------|--------------------|-------------------------------------------------------------------------------------------------|
| Day 1 | Foundation         | Next.js/Tailwind setup, Axios, Login, authentication, protected routes                          |
| Day 2 | Product Management | Product listing, responsive UI, pagination, loading/error/empty states                          |
| Day 3 | Search & Discovery | Search, debounce, category filter, sorting, URL state                                           |
| Day 4 | CRUD               | Product details, add, edit, delete, validation, confirmation                                    |
| Day 5 | Quality & Delivery | Race-condition handling, edge cases, testing, responsive fixes, README, Git cleanup, deployment |

## 18. Evaluation Framework

| Area                            | Weight |
|---------------------------------|--------|
| React / Next.js fundamentals    | 20%    |
| API integration                 | 15%    |
| Authentication                  | 10%    |
| Product functionality           | 15%    |
| Search / pagination / filtering | 15%    |
| Error & loading handling        | 10%    |
| Code quality / architecture     | 10%    |
| UI / responsiveness             | 5%     |
| Total                           | 100%   |

## 19. Interview / Code-Walkthrough Guidance

Because AI tools are permitted, completion alone should not be treated as proof of independent understanding. A short follow-up walkthrough is recommended.

- Ask the candidate to explain the search debounce implementation.
- Ask what happens when an older API response arrives after a newer request.
- Ask why URL state is useful and how it is synchronized.
- Ask why the Axios client is centralized.
- Ask the candidate to implement a small change live, such as adding a minimum-price filter.
- Ask them to explain one piece of code they generated or modified with AI.

## 20. Definition of Done

The assessment is complete when the deployed application implements the core requirements, handles normal loading/error/empty and edge states without crashing, has a clean enough structure to explain, contains meaningful Git history, and includes complete setup/documentation. Advanced items should be clearly documented if incomplete.

End of PRD